Brainbase Model Product — Implementation Plan

0. Product Definition

The product is a first-party Brainbase model interface that users call like a normal LLM.

Public-facing names should not use the word “router.”

Examples:

```
brainbase-chat
brainbase-code
brainbase-agent
brainbase-fast
brainbase-premium
brainbase-reasoning
```

From the user’s perspective:

```
client.chat.completions.create(
    model="brainbase-chat",
    messages=[...]
)
```

From Brainbase’s internal perspective, this is a modular routing runtime that decides which model, provider, cascade, workflow, or fine-tuned model should handle the request.

The product should be built so the **constant product shell remains stable**, while the changing parts are plug-ins:

```
Constant product shell:
  API
  protocol
  model resolver
  model registry
  policy registry
  Portkey executor
  OpenRouter passthrough
  Supabase traces
  benchmark runner
  shadow testing
  deployment
```

```
Changing parts:
  routing algorithms
  trained router artifacts
  fine-tuned models
  model strings
  benchmark datasets
```

The core engineering principle:

```
Build the product runtime first.
Make routers and algorithms plug-and-play later.
```

---

1. Product Goals

1.1 User-facing goals

The customer should be able to:

```
1. Call Brainbase like a normal LLM.
2. Use model names like brainbase-chat or brainbase-code.
3. Not care whether the backend uses Claude, GPT, Gemini, Qwen, DeepSeek, or a
Brainbase fine-tuned model.
4. Optionally select behavior through public model names.
5. Receive OpenAI-compatible responses.
```

1.2 Internal Brainbase goals

Brainbase should be able to:

```
1. Add any new model by adding a model string/config.
2. Add any new router algorithm by implementing a RouterPolicy plug-in.
3. Add a trained router by uploading a versioned artifact.
4. Run benchmarks across router policies.
5. Shadow-test new routers before promoting them.
6. Collect traces for future router training/fine-tuning.
7. Keep the product shell stable while algorithms evolve.
```

1.3 MVP success definition

The MVP is successful when:

```
A user can call:
  model="brainbase-chat"

Internally:
  request is normalized
  active RouterPolicy is loaded
  model candidates are resolved
  RoutePlan is produced
  RoutePlan is validated
  model is executed through Portkey/OpenRouter
  response is returned OpenAI-compatible
  trace is stored in Supabase
  benchmark runner can compare policies
  new routers can be added as plug-ins
  new model strings can be added through config
```

---

2. High-Level Architecture

```
Customer / Brainbase App
        ↓
OpenAI-Compatible API
        ↓
Brainbase Runtime Kernel
        ↓
RouterPolicy Plug-in
        ↓
RoutePlan
        ↓
RoutePlan Validator
        ↓
Model Resolver
        ↓
Portkey Executor
        ↓
OpenRouter / Direct Provider / Self-hosted Later
        ↓
Response Normalizer
```

```
        ↓
Supabase Traces + Benchmark Data
```

The public product is:

```
brainbase-chat
brainbase-code
brainbase-agent
```

The internal architecture is:

```
RouterPolicy
RoutePlan
ModelRegistry
PolicyRegistry
PortkeyExecutor
BenchmarkRunner
```

---

3. Core Design Decision

The product should not be built around one router.

Do not build:

```
HyDRA product
GraphRouter product
LLMRouter wrapper
OpenRouter router wrapper
```

Build:

```
Brainbase Model Runtime
  with swappable RouterPolicy plug-ins
```

The stable contract is:

```
RouterPolicy.plan(...) -> RoutePlan
```

Not:

```
RouterPolicy.select(...) -> selected_model
```

Why? Because future algorithms may do more than select one model. They may output:

```
single model
fallback list
cascade
multi-call plan
agent-step route
token budget
context selection
online-learning update
```

So the product must support `RoutePlan`, not just `selected_model`.

---

4. Repository Layout

Recommended monorepo structure:

```
brainbase-model-runtime/
  README.md
  pyproject.toml
  Dockerfile
  render.yaml
  .env.example

  app/
    main.py
    config.py
    dependencies.py

  api/
    openai_compatible.py
    models.py
    health.py
    internal.py

  protocol/
    router_request.py
    model_profile.py
```

```
    routing_context.py
    routing_budget.py
    route_plan.py
    route_step.py
    feedback_event.py
    schemas.py

  runtime/
    kernel.py
    request_normalizer.py
    public_model_resolver.py
    policy_loader.py
    model_candidate_resolver.py
    route_plan_validator.py
    fallback_manager.py
    response_normalizer.py

  registries/
    model_registry.py
    policy_registry.py
    tenant_registry.py
    model_registry.yaml
    policy_registry.yaml
    tenant_config.yaml

  policies/
    base.py

    always_strongest/
      policy.py
      tests.py

    always_cheapest/
      policy.py
      tests.py

    manual_rules/
      policy.py
      tests.py

    hydra/
      policy.py
      artifact_loader.py
      manifest.example.yaml
      tests.py

    graphrouter/
      policy.py
```

```
    artifact_loader.py
    manifest.example.yaml
    tests.py

  brainbase_trained/
    policy.py
    artifact_loader.py
    manifest.example.yaml
    tests.py

executor/
  portkey_executor.py
  openrouter_model_resolver.py
  execution_result.py
  errors.py

storage/
  supabase_client.py
  trace_repository.py
  registry_repository.py
  benchmark_repository.py
  artifact_repository.py

evals/
  benchmark_runner.py
  replay_runner.py
  policy_comparator.py
  scoring.py
  report_generator.py

scripts/
  run_local.py
  seed_registry.py
  benchmark_policy.py
  test_model_string.py
  add_model.py
  add_policy.py

tests/
  test_protocol.py
  test_route_plan_validator.py
  test_model_resolver.py
  test_runtime_kernel.py
  test_portkey_executor.py
```

---

5. Team / Ownership Boundaries

The product should be laid out so separate people can own separate areas.

5.1 API owner

Owns:

```
api/
app/
OpenAI-compatible request/response handling
public model names
streaming compatibility
auth hooks later
```

Does not own router algorithms.

5.2 Protocol/runtime owner

Owns:

```
protocol/
runtime/
RoutePlan schema
RoutePlan validator
runtime kernel
policy loading
model candidate resolution
```

This is the most important product shell owner.

5.3 Policy/plugin owner

Owns:

```
policies/
RouterPolicy implementations
HyDRA adapter
GraphRouter adapter
Brainbase-trained router adapter
baseline policies
```

This person should be able to add routers without touching the API or executor.

5.4 Executor/provider owner

Owns:

```
executor/
Portkey execution
OpenRouter model-string resolution
provider error handling
streaming execution
fallback execution
```

Does not own routing decisions.

5.5 Data/evals owner

Owns:

```
storage/
evals/
trace storage
benchmark runner
replay runner
policy comparison
training dataset exports
```

5.6 Deployment/infra owner

Owns:

```
Dockerfile
render.yaml
environment variables
Supabase project
deployment
secrets
logs
```

---

6. Public API Design

6.1 Chat completions endpoint

Expose:

```
POST /v1/chat/completions
```

Request:

```
{
  "model": "brainbase-chat",
  "messages": [
    {
      "role": "user",
      "content": "Write a follow-up email."
    }
  ]
}
```

Response:

```
{
  "id": "chatcmpl_...",
  "object": "chat.completion",
  "created": 1234567890,
  "model": "brainbase-chat",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "..."
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 123,
    "completion_tokens": 456,
    "total_tokens": 579
```

```
  }
}
```

6.2 Public model aliases

Initial aliases:

```
public_models:
  brainbase-chat:
    policy: always-strongest:v0
    model_pool: default
    mode: balanced

  brainbase-code:
    policy: always-strongest:v0
    model_pool: coding
    mode: code

  brainbase-agent:
    policy: always-strongest:v0
    model_pool: agent
    mode: agent

  brainbase-fast:
    policy: always-cheapest:v0
    model_pool: default
    mode: fast
```

In early MVP, these can all use baseline policies. Later they can map to HyDRA, GraphRouter, or Brainbase-trained policies.

---

7. Protocol Implementation

7.1 RouterRequest

Normalized form of the incoming user request.

```
@dataclass
class RouterRequest:
    request_id: str
    public_model: str
    messages: list[dict]
```

```
    tools: list[dict] | None
    response_format: dict | None
    modality: str
    tenant_id: str | None
    workflow_id: str | None
    step_id: str | None
    metadata: dict
```

7.2 ModelProfile

Represents a model that can be selected.

```
@dataclass
class ModelProfile:
    id: str
    executor: str
    executor_model: str
    provider: str
    status: str
    supports: dict
    limits: dict
    cost: dict
    capabilities: dict
    metadata: dict
```

Example ID format:

```
openrouter:anthropic/claude-sonnet-4.6
openrouter:openai/gpt-5
openrouter:google/gemini-pro
openrouter:qwen/qwen-coder
anthropic:claude-opus-latest
openai:gpt-latest
selfhosted:brainbase-qwen-agent-v1
```

7.3 RoutingContext

```
@dataclass
class RoutingContext:
    tenant_config: dict
    public_model_config: dict
    workflow_context: dict | None
```

```
    historical_performance: dict | None
    policy_config: dict
```

7.4 RoutingBudget

```python
@dataclass
class RoutingBudget:
    max_cost_usd: float | None
    max_latency_ms: int | None
    max_cascade_depth: int
    allow_multi_call: bool
    allow_expensive_models: bool
    mode: str
```

7.5 RoutePlan

```python
@dataclass
class RoutePlan:
    mode: str
    selected_model: str | None
    steps: list[RouteStep]
    fallback_models: list[str]
    confidence: float
    policy_name: str
    policy_version: str
    metadata: dict
```

Supported initial modes:

```
single
cascade
agent_step
```

Future modes:

```
multi_call
budgeted_single
context_routing
```

7.6 RouteStep

```
@dataclass
class RouteStep:
    model: str
    role: str | None
    condition: str | None
    max_tokens: int | None
    timeout_ms: int | None
    metadata: dict
```

---

8. RouterPolicy Plug-in Contract

Every router must implement:

```
class RouterPolicy:
    name: str
    version: str
    supported_modes: list[str]

    def plan(
        self,
        request: RouterRequest,
        candidates: list[ModelProfile],
        context: RoutingContext,
        budget: RoutingBudget
    ) -> RoutePlan:
        ...
```

8.1 Baseline example

```
class AlwaysStrongestPolicy(RouterPolicy):
    name = "always-strongest"
    version = "v0"
    supported_modes = ["single"]

    def plan(self, request, candidates, context, budget):
        model = max(
            candidates,
            key=lambda m: m.capabilities.get("overall", 0)
        )
```

```
        return RoutePlan(
            mode="single",
            selected_model=model.id,
            steps=[],
            fallback_models=[],
            confidence=1.0,
            policy_name=self.name,
            policy_version=self.version,
            metadata={
                "reason": "highest_overall_capability"
            }
        )
```

8.2 Future HyDRA example

```
class HydraPolicy(RouterPolicy):
    name = "hydra"
    version = "v1"
    supported_modes = ["single"]

    def plan(self, request, candidates, context, budget):
        required_capabilities = self.model.predict_capability_needs(request)
        selected = self.shortfall_match(required_capabilities, candidates,
budget)

        return RoutePlan(
            mode="single",
            selected_model=selected.id,
            steps=[],
            fallback_models=[],
            confidence=selected.confidence,
            policy_name=self.name,
            policy_version=self.version,
            metadata={
                "required_capabilities": required_capabilities
            }
        )
```

The runtime does not care how HyDRA works internally. It only cares that it returns a valid `RoutePlan`.

---

9. Model Resolver

The Model Resolver makes the “model string only” goal possible.

A router can output:

```
openrouter:anthropic/claude-sonnet-4.6
```

The resolver converts it into the executor format:

```
executor = portkey
executor_model = @openrouter/anthropic/claude-sonnet-4.6
```

9.1 Model registry entry

```
models:
  - id: openrouter:anthropic/claude-sonnet-4.6
    executor: portkey
    executor_model: "@openrouter/anthropic/claude-sonnet-4.6"
    provider: openrouter
    status: enabled
    supports:
      tools: true
      vision: true
      json: true
      streaming: true
    limits:
      context_window: 200000
    cost:
      input_per_million: null
      output_per_million: null
    capabilities:
      overall: 0.90
      reasoning: 0.88
      coding: 0.91
      debugging: 0.87
      tool_use: 0.90
```

9.2 Dynamic passthrough

The product should support dynamic testing of unregistered OpenRouter model strings.

Example:

```
openrouter:qwen/new-model
```

If not registered:

```
allow execution only in test/debug mode
mark as unverified
do not allow automatic production routing
log as dynamic_passthrough
```

This lets Brainbase test new model strings quickly while keeping production safe.

---

10. RoutePlan Validator

Before execution, every RoutePlan must be validated.

Validation checks:

```
selected model exists or is allowed dynamic passthrough
model is enabled for tenant
model supports requested tools/vision/json
model fits context length
plan fits cost budget
plan fits latency budget
cascade depth is allowed
fallback models are valid
route mode is supported
policy is allowed for public model
```

If validation fails:

```
fallback to safe default policy
or return controlled error
```

The validator protects the product from bad router plug-ins.

---

11. Portkey Executor

The executor receives a validated RoutePlan.

It should support:

```
single model execution
cascade execution
fallback execution
streaming
tool calling
JSON response format
provider error handling
cost/latency/tokens capture
```

11.1 MVP path

```
RoutePlan
  ↓
selected_model
  ↓
ModelResolver
  ↓
PortkeyExecutor
  ↓
OpenRouter model
```

11.2 Execution rule

Portkey executes the selected model. Portkey should not decide the route during benchmark mode.

For MVP:

```
Brainbase decides.
Portkey executes.
OpenRouter supplies models.
Supabase stores traces.
```

---

12. Supabase Data Model

Use Supabase for product state and traces.

12.1 Tables

public_model_aliases

Maps customer-facing model names to policies.

```
public_model_aliases (
  id uuid primary key,
  public_model text,
  policy_name text,
  policy_version text,
  model_pool text,
  mode text,
  config jsonb,
  created_at timestamp
)
```

model_registry

```
model_registry (
  id text primary key,
  executor text,
  executor_model text,
  provider text,
  status text,
  supports jsonb,
  limits jsonb,
  cost jsonb,
  capabilities jsonb,
  metadata jsonb,
  created_at timestamp,
  updated_at timestamp
)
```

policy_registry

```
policy_registry (
  id uuid primary key,
  name text,
```

```
  version text,
  type text,
  artifact_uri text,
  status text,
  supported_modes jsonb,
  config jsonb,
  created_at timestamp,
  updated_at timestamp
)
```

traces

```
traces (
  request_id text primary key,
  public_model text,
  tenant_id text,
  policy_name text,
  policy_version text,
  route_plan jsonb,
  selected_model text,
  candidate_models jsonb,
  fallback_used boolean,
  input_tokens int,
  output_tokens int,
  cost_usd numeric,
  latency_ms int,
  status text,
  error text,
  created_at timestamp
)
```

benchmark_runs

```
benchmark_runs (
  id uuid primary key,
  name text,
  policy_name text,
  policy_version text,
  dataset_name text,
  metrics jsonb,
  report jsonb,
  created_at timestamp
)
```

router_artifacts

```
router_artifacts (
  id uuid primary key,
  policy_name text,
  policy_version text,
  artifact_uri text,
  manifest jsonb,
  eval_report jsonb,
  status text,
  created_at timestamp
)
```

12.2 Storage

For MVP, use Supabase Storage for:

```
router artifacts
eval reports
benchmark datasets
training exports
```

This avoids needing S3 immediately.

S3 can be added later if artifacts/datasets become large.

---

13. Benchmark System

13.1 Why benchmark system is part of MVP

The product is useless internally if Brainbase cannot answer:

```
Is this new router better?
Is this new model worth enabling?
Should we promote this policy?
What is the cost/latency/quality tradeoff?
```

So the benchmark system is part of the constant product shell.

13.2 Benchmark runner

The benchmark runner should run:

```
policy A
policy B
policy C
```

against:

```
test prompts
historical traces
agent workflow examples
coding tasks
tool-use tasks
```

Metrics:

```
quality score
cost
latency
fallback rate
error rate
selected model distribution
cost per successful task
```

13.3 MVP benchmark policies

Initially compare:

```
always-strongest
always-cheapest
manual-rules
current-active-policy
new-candidate-policy
```

Later add:

```
HyDRA
GraphRouter
```

```
Avengers-style
Brainbase-trained
```

13.4 Benchmark output

```
{
  "policy_name": "hydra",
  "policy_version": "v1",
  "dataset": "brainbase-code-eval-v0",
  "quality_score": 0.91,
  "avg_cost_usd": 0.0042,
  "latency_p50": 1200,
  "latency_p95": 3900,
  "fallback_rate": 0.04,
  "win_rate_vs_always_strongest": 0.97,
  "cost_savings_vs_always_strongest": 0.38,
  "recommendation": "shadow_test"
}
```

---

14. Shadow Testing

Shadow testing lets Brainbase evaluate new routers on real traffic safely.

Flow:

```
real request comes in
  ↓
live policy produces RoutePlan and executes
  ↓
shadow policy also produces RoutePlan
  ↓
shadow plan is not executed by default
  ↓
compare decisions offline
```

Shadow trace:

```
{
  "request_id": "req_123",
  "live_policy": "always-strongest:v0",
  "live_selected_model": "openrouter:anthropic/claude-sonnet",
```

```
  "shadow_policy": "hydra:v1",
  "shadow_selected_model": "openrouter:qwen/qwen-coder",
  "live_cost_usd": 0.0042,
  "estimated_shadow_cost_usd": 0.0011
}
```

Later, shadow mode can optionally execute both outputs for smaller eval batches if quality comparison is needed.

---

15. Router Addition Workflow

When a new router algorithm comes out:

Step 1: Add policy folder

```
policies/new_router/
  policy.py
  artifact_loader.py
  manifest.example.yaml
  tests.py
```

Step 2: Implement RouterPolicy

```python
class NewRouterPolicy(RouterPolicy):
    name = "new-router"
    version = "v1"
    supported_modes = ["single"]

    def plan(self, request, candidates, context, budget):
        native_result = self.native_router.predict(...)
        return RoutePlan(...)
```

Step 3: Register policy

```
policies:
  - name: new-router
    version: v1
    type: research_router
    status: benchmark_only
    artifact_uri: supabase://router-artifacts/new-router/v1
```

Step 4: Run tests

```
schema tests
RoutePlan validation tests
budget tests
model support tests
executor compatibility tests
```

Step 5: Run benchmark

```
python scripts/benchmark_policy.py
  --policy new-router:v1
  --dataset default-eval-v0
  --baselines always-strongest,always-cheapest
```

Step 6: Shadow test

```
status: shadow
```

Step 7: Promote

```
status: enabled
```

No API changes should be needed.

---

16. Model Addition Workflow

When a new model comes out:

Step 1: Add model string

```
models:
  - id: openrouter:new-provider/new-model
    executor: portkey
    executor_model: "@openrouter/new-provider/new-model"
    provider: openrouter
    status: registered
```

Step 2: Test execution

```
python scripts/test_model_string.py
  --model openrouter:new-provider/new-model
```

Step 3: Calibrate

Run small internal evals:

```
chat
code
tool use
reasoning
latency
cost
```

Step 4: Add capability scores

```
capabilities:
  overall: 0.82
  reasoning: 0.84
  coding: 0.77
  tool_use: 0.80
  latency_score: 0.70
```

Step 5: Enable

```
status: enabled
```

Then all policies can consider it.

---

17. Implementation Phases

Phase 1: Skeleton API

Build:

```
FastAPI app
/v1/chat/completions
/v1/models
health endpoint
OpenAI-compatible response shape
public model alias config
```

Do not build advanced routing yet.

Phase 2: Protocol package

Build:

```
RouterRequest
ModelProfile
RoutingContext
RoutingBudget
RoutePlan
RouteStep
FeedbackEvent
schemas
validators
```

Phase 3: Registries

Build:

```
model registry loader
policy registry loader
public model alias loader
Supabase-backed registry storage
YAML fallback for local dev
```

Phase 4: Portkey/OpenRouter execution

Build:

```
Portkey client
OpenRouter model string resolver
single-model execution
streaming later
```

```
error handling
response normalization
```

Phase 5: Runtime kernel

Build:

```
request normalization
candidate resolution
policy loading
policy.plan() call
RoutePlan validation
execution
trace logging
```

Phase 6: Baseline policies

Build:

```
always-strongest
always-cheapest
manual-rules
random
```

This proves the shell works.

Phase 7: Supabase traces

Build:

```
trace table
policy table
model table
benchmark table
trace writer
trace viewer endpoint
```

Phase 8: Benchmark runner

Build:

```
dataset loader
policy runner
metric calculator
report generator
benchmark_runs storage
```

Phase 9: Shadow testing

Build:

```
shadow policy config
shadow decision logging
comparison report
promotion readiness
```

Phase 10: First serious router plug-in

Implement:

```
HyDRA-style capability router
```

Then optionally:

```
GraphRouter adapter
Avengers-style semantic router
Brainbase-trained router
```

Phase 11: Deployment

Deploy to Render.

Build:

```
Dockerfile
render.yaml
environment variable config
Supabase connection
Portkey/OpenRouter test
```

Phase 12: Documentation

Write:

```
customer SDK docs
internal policy plug-in docs
model addition docs
benchmark docs
deployment docs
```

---

18. First MVP Scope

The MVP should include:

```
OpenAI-compatible API
brainbase-chat / brainbase-code / brainbase-agent model names
RouterPolicy protocol
RoutePlan schema
Model Resolver
Model Registry
Policy Registry
Portkey Executor
OpenRouter passthrough
Supabase traces
Baseline policies
Benchmark runner
Shadow testing skeleton
Render deployment
```

The MVP should not include yet:

```
billing
rate limiting
LangFuse
Sentry unless desired
Kafka
direct OpenAI/Anthropic/Gemini keys
S3
Redis
full dashboard
```

```
fine-tuning pipeline
online learning
```

Those can be added later.

---

19. Deployment Plan

19.1 Recommended MVP deployment

Use:

```
Render
Supabase
Portkey
OpenRouter
GitHub
```

Why:

```
fast to ship
minimal infra
fewest keys
easy for coding agent
sufficient for MVP
```

19.2 Render service

Render should run the FastAPI backend.

Files:

```
Dockerfile
render.yaml
```

Example render.yaml:

```
services:
  - type: web
    name: brainbase-model-runtime
    env: docker
```

```
    plan: starter
    autoDeploy: true
    envVars:
      - key: PORTKEY_API_KEY
        sync: false
      - key: OPENROUTER_API_KEY
        sync: false
      - key: SUPABASE_URL
        sync: false
      - key: SUPABASE_SECRET_KEY
        sync: false
      - key: SUPABASE_PUBLISHABLE_KEY
        sync: false
```

19.3 Local development command

```
uvicorn app.main:app --reload --port 8000
```

19.4 Production command

```
uvicorn app.main:app --host 0.0.0.0 --port $PORT
```

---

20. Stacks and Third Parties Used

Core stack

```
Backend:
  Python
  FastAPI
  Pydantic
  Uvicorn

Execution:
  Portkey
  OpenRouter

Database:
  Supabase Postgres

Storage:
```

```
  Supabase Storage for MVP artifacts/eval reports

Deployment:
  Render

Version control:
  GitHub

Model access:
  OpenRouter through Portkey

Protocol:
  Brainbase RouterPolicy / RoutePlan internal protocol
```

Later optional stack

```
Direct providers:
  OpenAI
  Anthropic
  Google Gemini

Object storage:
  AWS S3 or Cloudflare R2

Event pipeline:
  Kafka

Cache/rate limit:
  Redis / Upstash

Training:
  Hugging Face
  W&B
  Modal / RunPod / CoreWeave

Observability:
  Portkey built-in first
  Sentry later if needed
  LangFuse not needed for MVP
```

---

21. External API Keys / Environment Variables Needed

This section lists only external keys that the implementer must provide. It does not include keys that the product can generate internally.

21.1 Required for MVP

```
PORTKEY_API_KEY=

OPENROUTER_API_KEY=

SUPABASE_URL=
SUPABASE_SECRET_KEY=
SUPABASE_PUBLISHABLE_KEY=
```

These are the only required third-party keys for the core MVP.

21.2 Required if coding agent must deploy to Render

```
RENDER_API_KEY=
```

If deployment is done manually through Render's UI, this key is not required.

21.3 Required if coding agent must access GitHub directly

```
GITHUB_TOKEN=
```

If the coding agent already works inside the repository or deployment is manually connected to GitHub, this may not be needed.

21.4 Optional later: S3 instead of Supabase Storage

Not needed for MVP if using Supabase Storage.

Use only when artifacts/datasets become large.

```
S3_BUCKET=
S3_REGION=
```

```
AWS_ACCESS_KEY_ID=
AWS_SECRET_ACCESS_KEY=
```

21.5 Optional later: direct provider lanes

Not needed for MVP because OpenRouter gives broad model access.

Add later for direct fallback, pricing, latency, or provider-specific features.

```
OPENAI_API_KEY=
ANTHROPIC_API_KEY=
GEMINI_API_KEY=
GOOGLE_API_KEY=
```

21.6 Optional later: Kafka

Not needed for standalone MVP.

Add when integrating deeply with Brainbase agent orchestration.

```
KAFKA_BOOTSTRAP_SERVERS=
KAFKA_API_KEY=
KAFKA_API_SECRET=
KAFKA_SCHEMA_REGISTRY_URL=
KAFKA_SCHEMA_REGISTRY_API_KEY=
KAFKA_SCHEMA_REGISTRY_API_SECRET=
```

21.7 Optional later: Redis / Upstash

Not needed for MVP.

Add for caching, locks, rate limits, or distributed coordination.

```
REDIS_URL=
UPSTASH_REDIS_REST_URL=
UPSTASH_REDIS_REST_TOKEN=
```

---

22. Final `.env.example`

MVP version

```
# Execution gateway
PORTKEY_API_KEY=

# Broad model access
OPENROUTER_API_KEY=

# Supabase
SUPABASE_URL=
SUPABASE_SECRET_KEY=
SUPABASE_PUBLISHABLE_KEY=
```

MVP + automated deployment

```
# Execution gateway
PORTKEY_API_KEY=

# Broad model access
OPENROUTER_API_KEY=

# Supabase
SUPABASE_URL=
SUPABASE_SECRET_KEY=
SUPABASE_PUBLISHABLE_KEY=

# Deployment
RENDER_API_KEY=

# Repo access, only if needed
GITHUB_TOKEN=
```

Future expanded version

```
# Core
PORTKEY_API_KEY=
OPENROUTER_API_KEY=
SUPABASE_URL=
SUPABASE_SECRET_KEY=
SUPABASE_PUBLISHABLE_KEY=
```

```
# Deployment
RENDER_API_KEY=
GITHUB_TOKEN=

# Optional direct providers
OPENAI_API_KEY=
ANTHROPIC_API_KEY=
GEMINI_API_KEY=
GOOGLE_API_KEY=

# Optional S3
S3_BUCKET=
S3_REGION=
AWS_ACCESS_KEY_ID=
AWS_SECRET_ACCESS_KEY=

# Optional Kafka
KAFKA_BOOTSTRAP_SERVERS=
KAFKA_API_KEY=
KAFKA_API_SECRET=
KAFKA_SCHEMA_REGISTRY_URL=
KAFKA_SCHEMA_REGISTRY_API_KEY=
KAFKA_SCHEMA_REGISTRY_API_SECRET=

# Optional Redis
REDIS_URL=
UPSTASH_REDIS_REST_URL=
UPSTASH_REDIS_REST_TOKEN=
```

---

23. Final Build Instruction to Implementer

Build the product as a stable Brainbase model runtime, not as one router implementation.

The first deliverable is not HyDRA or GraphRouter.

The first deliverable is:

```
OpenAI-compatible Brainbase model API
+
RouterPolicy / RoutePlan protocol
+
model-string resolver
+
```

```
Portkey/OpenRouter executor
+
Supabase registry/traces
+
benchmark and shadow-test skeleton
```

Once that is complete, router algorithms become plug-ins.

The product is complete when:

```
New model:
  add string/config → test → calibrate → enable

New router:
  implement RouterPolicy.plan() → benchmark → shadow → promote

New public Brainbase model:
  map public name to policy + model pool

Customer:
  calls brainbase-chat like a normal model
```

That is the plan.